

Custody

Proving digital evidence has not been changed

Technical write-up. Team Captain. ICSC 2026 Universities Hackathon, Track H.

1. Overview

Custody is a chain of custody system for digital evidence. It records a cryptographic fingerprint of an evidence file at the point of collection, maintains an append only log of every subsequent transfer, and verifies on demand whether the file or its handling record has changed. Verification results are presented as a single page report intended for a reader without a technical background.

The system operates without network access during collection, runs on a single machine, and uses only open source components. It comprises three deployable units.

Field application	React Native and Expo SDK 57. Collection, on device chunked hashing, a local SQLite queue and deferred synchronisation.
Backend service	Express 5 and PostgreSQL 16. Custody chain storage, verification, and report generation.
Dashboard	Next.js 15 and TypeScript. Case and item views, custody timeline, chunk map and verification controls.

The field application communicates with the backend over the local network. The service binds to all interfaces to permit this.

2. Integrity model

Two properties are verified independently: whether the evidence file matches the fingerprint recorded at collection, and whether the custody record itself has been modified. The two are reported as separate results throughout the system.

File integrity	A SHA-256 Merkle root computed over the file contents at collection, recomputed from storage on each verification and compared.
Item chain	Each custody event stores the SHA-256 hash of the preceding event. Modifying an event invalidates all subsequent hashes; verification reports the sequence number of the first failure.
Case chain	One event per item creation, holding the item reference as text rather than as a foreign key. Deleting an item therefore leaves the creation record intact and the absence detectable.

Event hashes are computed over a fixed field order and separator:

```
eventHash      = sha256( prevHash | itemId | actorId | action | deviceTime | fileHash )
caseEventHash  = sha256( prevHash | caseId | action | itemRef | itemRootHash | deviceTime )
```

Verification recomputes every hash from the stored field values of each event. Stored hash values are compared against the recomputed result and are not used as inputs. Two conditions are evaluated at each position: the recomputed hash matches the stored hash, and the stored predecessor hash matches the preceding event's hash. The first position at which either condition fails is returned with a reason code, which the dashboard uses to render the break at the corresponding entry.

Custody events are append only. UPDATE and DELETE on the events table are rejected by a database trigger rather than by application logic, so the constraint also applies to direct database connections. Corrections are recorded as additional events referencing the corrected event.

3. Chunked hashing

Files are processed in fixed 4 MB chunks. Each chunk is hashed independently and a binary Merkle tree is constructed over the ordered chunk hashes, with unpaired nodes promoted unchanged to the next level. The tree root is stored as the item fingerprint and the ordered chunk hashes are retained alongside it. The design yields three properties.

Constant memory. The complete file is never resident. Measured peak heap growth while hashing a 30 MB input was below 1 MB.

Change localisation. A single byte modification at offset 21,000,000 of a 30 MB file resolves to chunk 6 of 8, byte range 20,971,520 to 25,165,823. Verification returns the affected chunk indices and the corresponding byte range rather than a boolean result.

Resumability. Chunk hashes are independent, so verification of a partially processed file can be resumed.

Testing was performed at 30 MB. Behaviour at larger sizes is inferred from the memory profile rather than measured; a 100 GB input was not processed.

The field application and the backend implement the same scheme and must produce identical output. Two rules define the format: a chunk hash is SHA-256 over the raw chunk bytes, and every other hash is SHA-256 over a UTF-8 string of hexadecimal digits. On device, chunks are read through a seekable file handle; on the server, through a read stream with the running hash carried across buffer boundaries. Equivalence is asserted by a dedicated test suite.

4. Collection and synchronisation

Collection executes entirely on the device: the file is hashed, the record is written to local SQLite storage, and a collection event is queued. No network operation occurs in this path. Synchronisation runs separately, is batched, and resumes automatically when the service becomes reachable.

Evidence files are not transmitted. The client sends the Merkle root, the ordered chunk hashes, file size, collector identity and timestamps. This removes any dependency on request size limits and reflects the separation between an exhibit and its documentation.

Two timestamps are recorded for each item: the device clock at collection and the server clock at receipt. Both are stored and both appear in the report, including the interval between them. Device clocks are not independently verifiable and are not presented as authoritative.

A device cannot determine the storage layout of the evidence store, so a synchronised item carries no storage path until the exhibit is deposited through a separate endpoint, which records the path and appends a custody event. Verification distinguishes four states.

intact	The stored file matches the fingerprint recorded at collection.
altered	It does not. Affected chunk indices and byte range are returned.
awaiting_file	Fingerprint recorded, exhibit not yet deposited.
missing	The exhibit was deposited and is no longer present in storage.

5. Reporting

The report is a single page and is generated server side as a PDF. It states whether the item is unchanged, how that was established, who handled it and in what order, and whether the handling record could have been altered. Hash values and technical parameters are confined to an appendix.

Enumeration values and ISO 8601 timestamps are mapped to natural language before rendering. Automated tests assert that neither form appears in the report or in the dashboard.

6. Testing

240 assertions across six suites. Assertions cover response field names as well as behaviour, so an interface change between components is detected by the suite rather than at runtime.

Hash parity	13 assertions. The field application's hashing implementation is reimplemented in Node and compared against the backend across real files and tree widths of 1, 2, 3, 5, 7, 8 and 9 chunks.
API	104 assertions. All endpoints, response contracts, error handling, malformed input and injection attempts in path parameters.
Synchronisation	47 assertions across two suites. Offline collection and deferred sync, one suite importing the field application's own payload construction module and exercising it against the running service.
Dashboard	53 assertions against served markup, including the absence of enumeration values and ISO timestamps, and correct rendering of a broken chain.
Acceptance	23 assertions. End to end execution including deliberate file and record modification, item deletion, report generation and state reset.

7. Test data

All data in the system is synthetic and was generated for this project. It contains no real persons, cases, accounts, records or personal data. Names, badge numbers and account numbers are invented. The stand-in for a device extraction is 30 MB of random bytes; the system operates on byte sequences and does not interpret file contents. Provenance is documented in the repository.

8. Constraints

Pre-collection tampering	Modification before the fingerprint is recorded produces a valid chain over altered evidence. The system establishes integrity from collection onward only.
Device clock	Offline timestamps originate from the device and may be incorrect. Both clocks are recorded; the device value is not treated as authoritative.
Scale	Verified at 30 MB. Larger inputs are inferred from the memory profile.
Event attribution	Events are attributed to an authenticated identity, not to a cryptographic key held by an individual. Per officer signing is not implemented.
File retention	The system detects modification and deletion. It does not prevent either.
Privileged access	An actor with full database and server access could reconstruct a consistent chain. Periodic publication of root hashes to an external location would mitigate this and is not implemented.

A distributed ledger was considered and rejected. Such systems address the absence of a shared trusted authority between parties. In the target settings that authority exists, and a hash chain with append only storage provides equivalent tamper evidence at lower operational cost.

9. Running the system

```
brew services start postgresql@16      # or: sudo service postgresql start
cd hackathonBackend && npm install && npm run demo:reset && npm run dev      # API   :4000
cd hackathonWebApp   && npm install && npm run dev                          # web   :3000
npm test && npm run test:web && npm run test:acceptance                      # 240 assertions
```

Setup instructions, the demonstration procedure and the testing guide are included in the repository as README.md, DEMO.md and TESTING.md.